

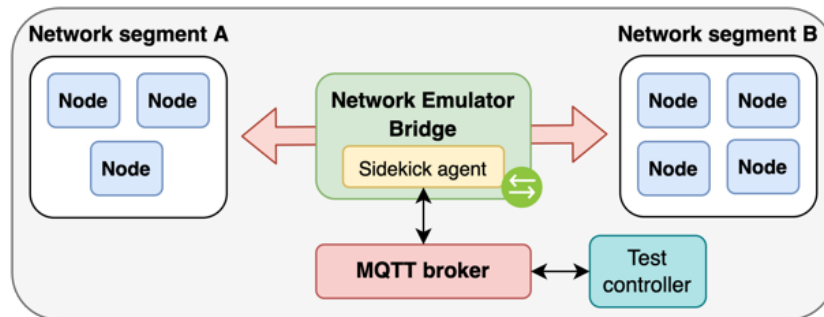
Sidekick tool for Traffic Shaper integration

History

Version	Author	Description/Changelog
1.0	T. Cruz	Original version
2.0	T.Cruz	New functionality: config reloads, logging, runtime test config reloads, config query
3.0	T. Cruz	Multi group support, with multiple separate profiles, JSON format revision
3.1	T. Cruz	Correction of typo in the daemon mode parameter description
3.2	T. Cruz	Minor updates, with topology figure being introduced

Capabilities

The sidekick tool was written for the purpose of introducing communications-related disturbances in a controlled way, to test/evaluate the performance and robustness of communications protocols, APIs or services. The fundamental operation model for this tool is presented on the next figure.



A Network Emulator bridge based on a Linux virtual appliance provides the means to constrain/disturb the traffic between communicating peers on different network segments, in a transparent way. This appliance is configured with three network interfaces: two for the transparent bridge and one for out-of-band experiment control. Network traffic shaping capabilities are provided by the native Linux *TC* and *Netem* subsystems, to constrain bandwidth or inject disturbances such as packet losses, jitter, or latency.

To deal with the integration of these capabilities within a fault injection framework, the sidekick agent provides remote control of the traffic shaper, for integrated experiment management purposes. This tool provides the following capabilities:

The tool provides the following capabilities:

- Remote scheduling for time-triggered network shaper profile activation
- Integration with standard MQTT brokers
- Easy-to-use JSON profile configuration file
- Can operate in foreground or in daemonized/background mode
- Trigger precision around 0.02s, on average (for overall system load <20%)

Tool Profile Configuration and execution

1. Configuration

On the `sidekick.py` file, the following configurations must be manually adjusted.

```
#USER CONFIGURARIONS
PATH="<deployment path>"
broker = 'MQTT BROKER'
port = 1883
topic_send = "netem/status"
topic_recv = "netem/scheduler"
```

The PATH variable should be adjusted to the location of the sidekick files. Sudo permissions must be granted.

It is assumed that the machine used for the transparent bridge has 3 network interfaces: eth0, eth1, and eth2. The first two are used for the bridge, with eth2 being used for management.

2. Execution

All the files are installed on the PATH folder, on the *Emunet* host. Users must access with the provided credentials, switch to root ("sudo -i") and go to the tool folder. The complete steps are illustrated below:

```
(tjcruez@emunet)~[~]
$ sudo -i
(Message from Kali developers)

We have kept /usr/bin/python pointing to Python 2 for backwards
compatibility. Learn how to change this and avoid this message:
⇒ https://www.kali.org/docs/general-use/python3-transition/

(Run "touch ~/.hushlogin" to hide this message)
(root@emunet)~[~]
# cd /home/tjcruez/MQTT_Server
(root@emunet)-[/home/tjcruez/MQTT_Server]
# ./sidekick.py
Connected to MQTT Broker!
```

The tool can run in foreground and background modes, accordingly with the specific needs.

1. **To execute the tool in foreground mode**, just type `./sidekick.py`, as shown below (the date +%s command was added to get the current time in epoch/UNIX format):

```
(root@emunet)-[/home/tjcruez/MQTT_Server/WIP]
# date +%s; ./sidekick.py
1632925125
Connected to MQTT Broker!
Received request:01B:1632925145:30
START EPOCH:1632925145.0
START SCHEDULED FOR:time.struct_time(tm_year=2021, tm_mon=9, tm_mday=29, tm_hour=14, tm_min=19,
END SCHEDULED FOR:time.struct_time(tm_year=2021, tm_mon=9, tm_mday=29, tm_hour=14, tm_min=19, tm
ON CURRENT EPOCH 01B:1632925145.0114062:1632925145.031563
ON CURRENT TIME 01B:time.struct_time(tm_year=2021, tm_mon=9, tm_mday=29, tm_hour=14, tm_min=19,
OFF CURRENT EPOCH 01B:1632925175.0300612:1632925175.050264
OFF CURRENT TIME 01B:time.struct_time(tm_year=2021, tm_mon=9, tm_mday=29, tm_hour=14, tm_min=19,
End scheduled batch for group 01B:1632925175.0505438
```

In foreground mode, all status messages are displayed to the console, for debugging purposes. Information about scheduled tests (START and END times) and executed tests (ON and OFF times) are provided.

2. **To execute the tool in background (daemon) mode**, type:

```
./sidekick.py --daemon
```

In background mode, all debug information is recorded to syslog.

```
(root@emunet)~# cat /var/log/syslog | grep sidekick
Sep 29 15:21:06 emunet sidekick.py: Connected to MQTT Broker!
Sep 29 15:21:16 emunet sidekick.py: Received request:01B:1632925284:30
Sep 29 15:21:16 emunet sidekick.py: START EPOCH:1632925284.0
Sep 29 15:21:16 emunet sidekick.py: START SCHEDULED FOR:time.struct_time(tm_year=2021, tm_mon=9, tm_mday=29,
day=272, tm_isdst=0)
Sep 29 15:21:16 emunet sidekick.py: END SCHEDULED FOR:time.struct_time(tm_year=2021, tm_mon=9, tm_mday=29, tr
y=272, tm_isdst=0)
Sep 29 15:21:24 emunet sidekick.py: ON CURRENT EPOCH 01B:1632925284.0078845:1632925284.0288885
Sep 29 15:21:24 emunet sidekick.py: ON CURRENT TIME 01B:time.struct_time(tm_year=2021, tm_mon=9, tm_mday=29,
day=272, tm_isdst=0)
Sep 29 15:21:54 emunet sidekick.py: OFF CURRENT EPOCH 01B:1632925314.02142:1632925314.0424984
Sep 29 15:21:54 emunet sidekick.py: OFF CURRENT TIME 01B:time.struct_time(tm_year=2021, tm_mon=9, tm_mday=29,
yday=272, tm_isdst=0)
Sep 29 15:21:54 emunet sidekick.py: End scheduled batch for group 01B:1632925314.0425987
```

3. Configuration

The network profiles used for the nominal and test conditions are described in the `setup.json` file, for which an example is shown below:

```
{
  "groups": [
    {
      "id": "01A",
      "address_pairs": [
        {
          "ip_left": "192.168.1.1",
          "ip_right": "192.168.1.2"
        },
        {
          "ip_left": "192.168.1.3",
          "ip_right": "192.168.1.2"
        },
        {
          "ip_left": "192.168.1.5",
          "ip_right": "192.168.1.6"
        }
      ],
      "configUP_nom": [
        {
          "bw": "50000kbit",
          "jitter": "",
          "loss": "0%",
          "latency": "0ms"
        }
      ],
      "configDOWN_nom": [
        {
          "bw": "50000kbit",
          "jitter": "",
          "loss": "0%",
          "latency": "0ms"
        }
      ],
      "configUP_test": [
        {
          "bw": "150kbit",
          "jitter": "",
          "loss": "1%",
          "latency": "10ms"
        }
      ],
      "configDOWN_test": [
        {
          "bw": "150kbit",
          "jitter": "",
          "loss": "1%",
          "latency": "10ms"
        }
      ]
    },
    {
      "id": "01B",
      "address_pairs": [
        {
          "ip_left": "192.168.1.7",
          "ip_right": "192.168.1.8"
        },
        {
          "ip_left": "192.168.1.9",
          "ip_right": "192.168.1.10"
        }
      ],
      "configUP_nom": [
        {
          "bw": "50000kbit",
          "jitter": "",
          "loss": "0%",
          "latency": "0ms"
        }
      ],
      "configDOWN_nom": [
        {
          "bw": "50000kbit",
          "jitter": "",
          "loss": "0%",
          "latency": "0ms"
        }
      ],
      "configUP_test": [
        {
          "bw": "150kbit",
          "jitter": "",
          "loss": "1%",
          "latency": "10ms"
        }
      ],
      "configDOWN_test": [
        {
          "bw": "150kbit",
          "jitter": "",
          "loss": "1%",
          "latency": "10ms"
        }
      ]
    }
  ]
}
```

This file is loaded by the *sidekick* tool as soon as it starts, activating the nominal upload and download configurations (_nom suffix entries) for all the identified IP address pairs. The file is organized by host groups, as such:

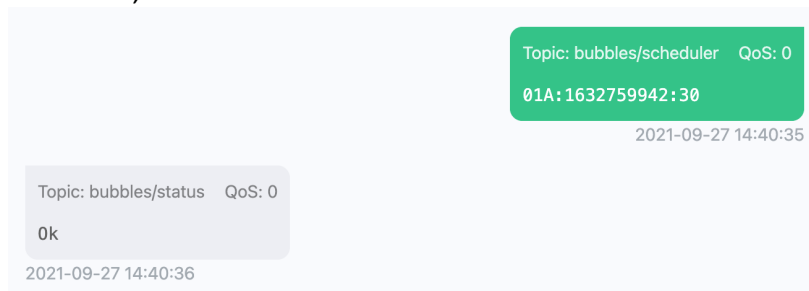
- Each IP group has an unique **ID**. ID must be a string identifier, unique to the specific group of IP pairs.

- The **address_pairs** entries define all the IP address pairs that are part of the experiment. For each pair, the tool will take care of activating and deactivating the nominal and test profiles, accordingly with the defined schedule. The “Left” and “Right” identifiers are associated with the network topology that was created – this is important to distinguish the UL/DL directions.
- The **config{UP,DOWN}_{nom,test}** entries are used to define the Upload (UL) and Download (DL) network profiles for the nominal state and for the test window. The entries are self-explanatory: units must be added.

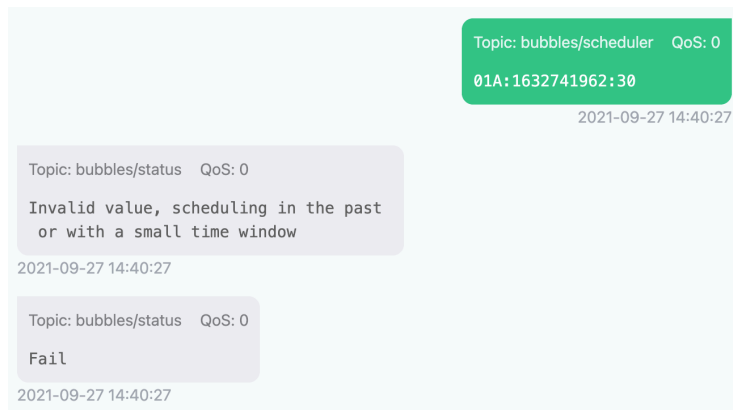
Remote Tool Management and Control (via MQTT)

The MQTTX application was used to interact with the **sidekick** tool, for demonstration purposes (and to obtain the following screenshots). There are equivalent MQTT tools, for different OS platforms. **The main rationale is for the fault injector configuration app to talk with the *sidekick* tool using this method**, posting requests to the “**bubbles/scheduler**” topic and getting responses via the “**bubbles/status**” topic. For this purpose, an MQTT server was installed and configured for the testbed (reachable via address 10.254.0.250, port 1883).

The **sidekick** tool was designed to allow for experiments to be scheduled, by supplying a starting moment for the test, encoded in UNIX timestamp/epoch format (UTC) and the test duration (in seconds). This also means that special care must be taken regarding the configuration of proper time synchronization across the machines involved in the experiment. For instance, to activate a scheduled test, an MQTT plaintext message has to be sent to the platform MQTT server, to the topic “**bubbles/scheduler**”, with the format “groupID:start:duration”, as shown below:



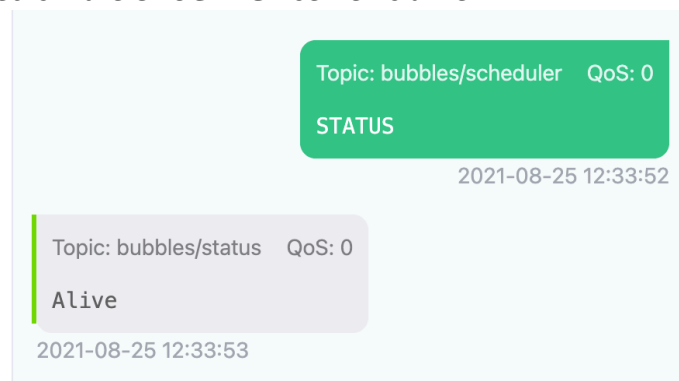
Status responses are provided in the topic “**bubbles/status**”. If the setup parameters are incorrect, the system will report an error.



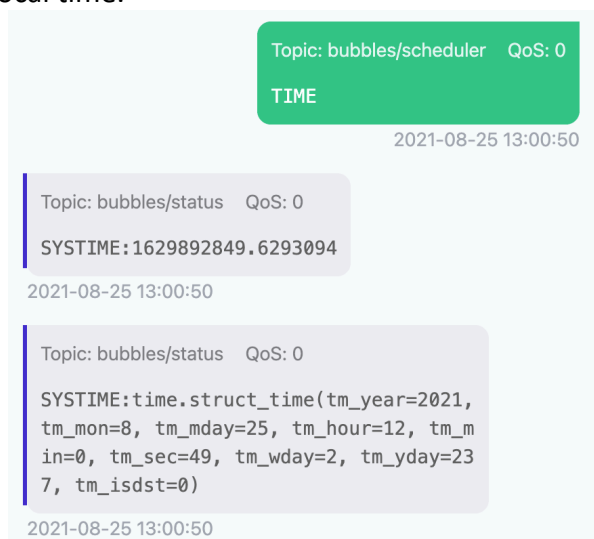
Also, if a new experiment is scheduled while a pending experiment already exists, the latter will be replaced by the new one (except if the parameters are invalid, in which case the old experiment will be kept in queue).

To terminate the tool, the **TERM** command can be issued via MQTT. This will terminate the running instance. Some other useful commands also exist:

- **STATUS:** checks if the *sidekick* server is alive.

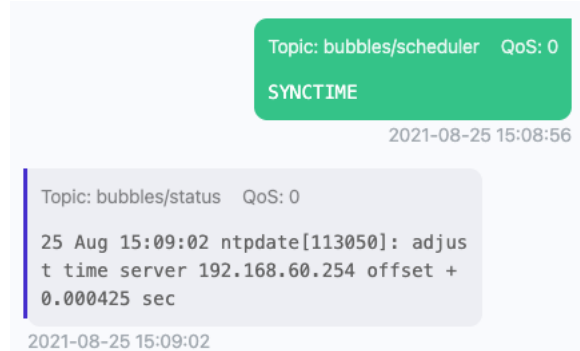


- **TIME:** reports local time.



- **TESTMODE(groupID):** manually activates test profile conditions for a specific group ID, for debugging purposes.

- **NORMMODE(groupID)**: manually reinstates nominal profile conditions for a specific group ID, for debugging purposes.
- **SYNCTIME**: synchronizes time with NTP server `ntp.dei.uc.pt`.



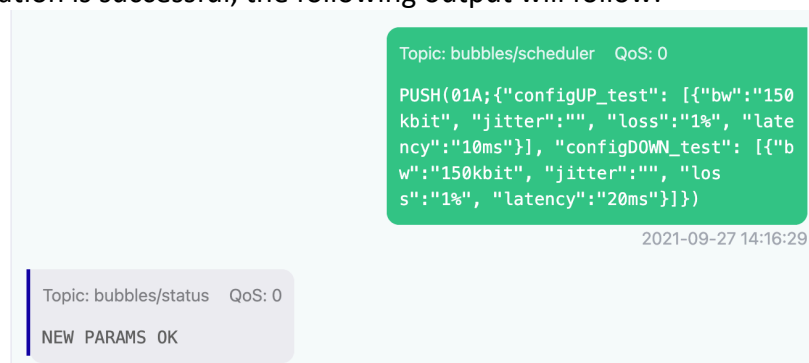
Bear in mind that, since v3.0, the VM that host the Network Emulator has a hypervisor-synchronized clock (and the hypervisor, by itself, synchronizes with the department's NTP server). Nevertheless, this command may be used to correct minor skewing.

- **RELOAD**: if you change the configuration file, issuing RELOAD will reload the complete configuration set. Configurations will be loaded but the command doesn't reset the nominal state settings to avoid creating unnecessary disruptions. The rationale for doing this is the assumption that the changes will almost always affect the test parameters, and it makes no sense to radically reset the configuration.
- **PUSH(groupID;<config>)**: allows to push new test profiles for a specific (existing) group, over-the-wire, via MQTT messages with JSON format. For example, any of the following messages will change the test configuration parameters, via MQTT (the parser is smart enough to deal with formatting):

```
PUSH(01A;{"configUP_test": [{"bw":"10kbit", "jitter":"", "loss":"1%", "latency":"10ms"}], "configDOWN_test": [ {"bw":"10kbit", "jitter":"", "loss":"1%", "latency":"10ms"}]})
```

```
PUSH(01A;{"configUP_test": [{"bw":"150kbit", "jitter":"", "loss":"1%", "latency":"10ms"}], "configDOWN_test": [{"bw":"150kbit", "jitter":"", "loss":"1%", "latency":"10ms"}]})
```

If the operation is successful, the following output will follow:



- **CONFIG(groupID)**: dumps the current nominal and test configuration profiles.

The screenshot shows two MQTT messages. The top message is a green box with the text: "Topic: bubbles/scheduler QoS: 0" and "CONFIG(01A)" with a timestamp of "2021-09-29 15:09:06". The bottom message is a light blue box with the text: "Topic: bubbles/status QoS: 0", followed by configuration details for NOMINAL UP, NOMINAL DOWN, TEST UP, and TEST DOWN, each with bandwidth, jitter, loss, and latency settings. It also has a timestamp of "2021-09-29 15:09:06".

- **LOG2SYSLOG, LOG_OFF**: respectively enable and disable logging of all status and experiment trigger messages to syslog (/var/log/syslog). Such entries can be easily parsed searching by the “sidekick” string, as shown in the next snapshot:

```
(root@emunet) - [ /home/tjcruz/MQTT_Server ]
# cat /var/log/syslog | grep sidekick
Sep  6 01:30:05 emunet sidekick.py: Received request:CONFIG
Sep  6 01:30:11 emunet sidekick.py: Received request:STATUS
Sep  6 01:30:25 emunet sidekick.py: Received request:TESTMODE
Sep  6 01:30:29 emunet sidekick.py: Received request:NORMMODE
Sep  6 01:30:33 emunet sidekick.py: Received request:TERM
Sep  6 01:30:33 emunet sidekick.py: Terminating
```

Useful notes and remarks

Keep time in sync!! Also, if you need to know the present time in epoch/UNIX format, type “date +%s” in a command prompt. Note that this uses a UTC reference. **SYNCTIME** is your friend!

Keep the number of IP pairs to the absolute minimum. When the *sidekick* tool switches between modes, it sequentially executes all the necessary commands. Since this is a sequential process, it means that the IP address pairs will not be affected at the precise same time. This can be slightly improved, but never completely solved.

When executed in foreground, the tool can also be terminated via CTRL+C. Also, the **TESTMODE** and **NORMMODE** commands must not be used together with scheduled tasks!